

CareerNavigator AI: Detailed Technical Project Status

Generated on: March 2026

1. Executive Summary

CareerNavigator AI is a decoupled MERN-AI Hybrid application intended to provide vocational alignment and explainability using the RIASEC model and LLM reasoning. The core infrastructure is complete, connecting a React frontend with a Node.js backend, and leveraging a dual-tier AI microservice architecture capable of both local and cloud-based inference.

2. Frontend SPA (React + Vite)

The User Interface is built as a Single Page Application (SPA), utilizing modern tooling for rapid development and high performance.

Core Technologies: React 18, Vite, Tailwind CSS, React Router DOM v7, Chart.js.

2.1 Routing & Navigation (App.jsx)

The application implements client-side routing with comprehensive views for the user journey.

- `/` (`Home.jsx`): Public landing page.
- `/login` & `/register` (`Login.jsx` , `Register.jsx`): JWT-based authentication flows.
- `/dashboard` (`Dashboard.jsx`): Protected user hub for assessment initiation and profile review.
- `/assessment` (`Assessment.jsx`): The 30-question RIASEC psychometric survey interface.
- `/recommendations` (`Recommendations.jsx`): Displays the ranked list of career matches.
- `/career/:id` (`CareerDetail.jsx`): Deep-dive into a specific career, visualizing alignment metrics (RVI, SHAP).
- `/favourites` (`Favourites.jsx`): User's bookmarked career paths.

Note: Minor cleanup required in `App.jsx` due to duplicate component imports (`Recommendations`, `Favourites`).

3. API Gateway & Backend Node.js

The Express API manages authentication, database routing, and proxying requests to the AI service.

Core Technologies: Node.js, Express.js, Mongoose, JWT, bcryptjs, mongodb-memory-server.

3.1 Advanced Database Resilience (server.js)

The backend features an intelligent fallback mechanism for database connectivity:

- Primary:** Connects to MongoDB Atlas via `MONGO_URI`.
- Secondary (Zero-Config Local Audit):** If the Atlas connection fails (e.g., due to IP whitelisting or missing credentials in non-production environments), the server automatically spins up an in-memory MongoDB instance (`mongodb-memory-server`).
- Auto-Seeding:** Upon falling back to the in-memory DB, it automatically seeds 30 RIASEC questions and loads the `careers_data.json` repository to ensure the app remains fully functional for demonstration and testing.

3.2 Implemented API Routes

- `POST /api/auth/register` & `POST /api/auth/login` : User management.
 - `GET /api/assessment` : Supplies the seeded psychometric questions.
 - `POST /api/careers/match` : The primary integration point that bridges the MERN stack to the Python AI microservice.
-

4. Dual-Tier AI Microservice

The Python backend contains sophisticated logic for multi-modal AI inference. Initially designed around local inference, it also maintains cloud-fallback capabilities.

4.1 Local Explainable AI Pipeline (`inference.py`)

This represents the primary zero-cost, local-first architecture.

- **Vector Storage:** Uses ChromaDB (`db_storage/chroma.sqlite3`) for persistent HNSW search.
- **Embeddings:** Employs `sentence-transformers` (`all-MiniLM-L6-v2`) to turn textual narratives into vectors.
- **Metrics Engine:**
 - **RVI (Resilience-Volatility Index):** Calculates automation risk based on O*NET data weights (e.g., Conventional=0.8 risk, Social=0.1 risk).
 - **SHAP Proxy Generation:** Computes math-based feature importance arrays to explain *why* a user matched a career.
- **Local LLM Integration:** Communicates via HTTP (`requests`) with a local **Ollama** container running `llama3:8b` to generate a 3-sentence encouraging, context-aware explanation based on RVI and SHAP scores.

4.2 Cloud Fallback Integration (`main.py`)

A FastAPI alternative endpoint designed for rapid prototyping or environments where local LLM inference is impossible.

- Proxies requests to **OpenAI GPT-3.5 Turbo** for career explanations.
 - Utilizes Scikit-Learn's `cosine_similarity` for quick RIASEC vector matching from memory-loaded `CareerProfile` objects.
-

5. Deployment Architecture

- **Vercel Readiness:** `vercel.json` is configured to handle SPA routing overrides (`"rewrites": [{"source": "/(.*)", "destination": "/"}]`), ensuring deep-links in React Router function correctly in production.
- **Concurrent Dev Environment:** `package.json` at the root uses `concurrently` to boot the React Vite server and the Node Express server simultaneously via `npm start` .

Next Steps / Pending Audit

- Resolve minor duplicate imports in `frontend/src/App.jsx` .
- Solidify environment variables for the production MongoDB deployment.
- Confirm integration of the two AI service entry points (`inference.py` vs FastAPI `main.py`).